Smart Cliff Learning Solutions

COE 2026-EXPLEO-KIOT-SDET-III-Assessment - 14

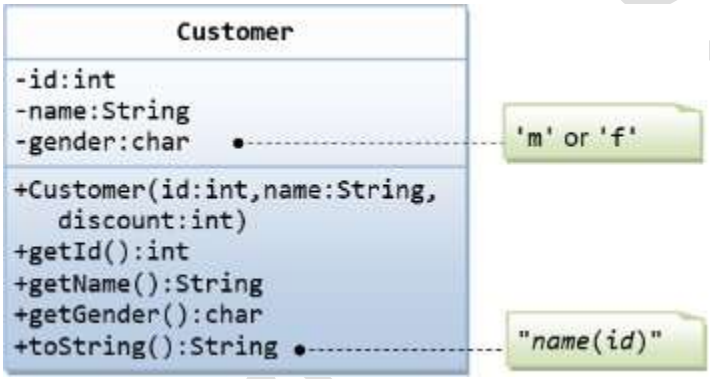
Module	**:**	**Java Fundamentals - OOP**
Date	**:**	**06/03/2026**
Time	**:**	**02:30 to 04:00**
Duration	**:**	**2:30 Hrs.**
Total Marks	**:**	**40 Marks**

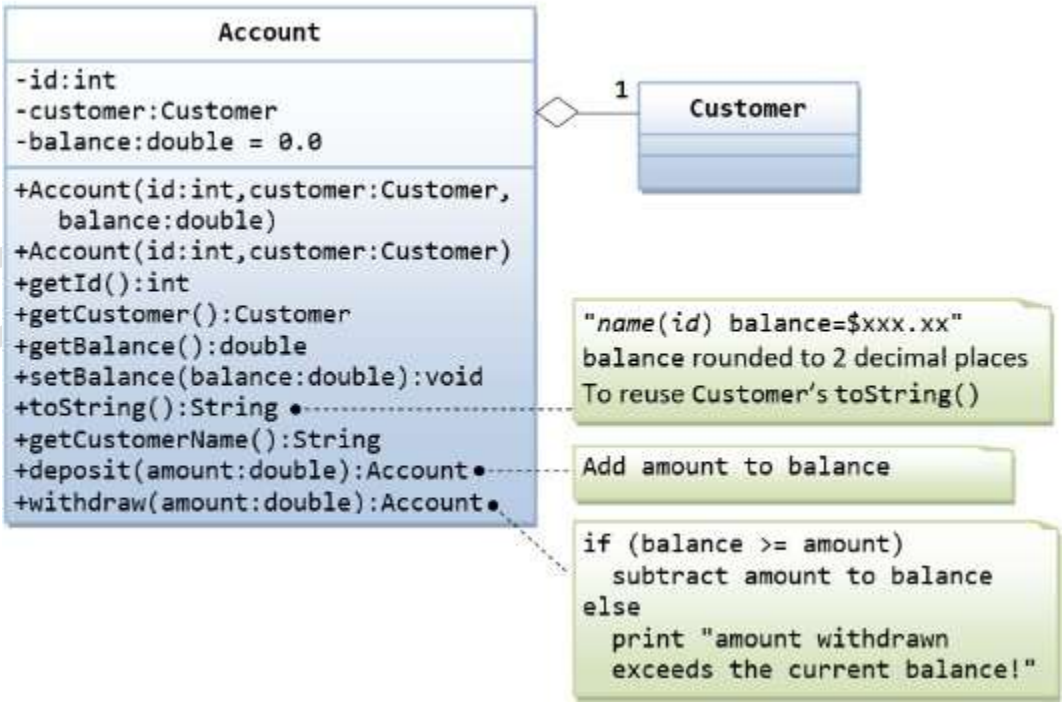
PART A: MCQ (15 x 1 : 15 MARKS)

PART B: PROGRAMMING(25 MARKS)

1. **CUSTOMER AND ACCOUNT (10 Marks)**

You are tasked with implementing a class called **Customer and account** were account has customer as one of its class member. Design the customer and account class as shown in the class diagram and test the methods of the both the classes in the driver class.





SmartCliff

Sample Input and Output:

ACCOUNT DETAILS

ACCOUNT_ID	3456
CUSTOMER_ID	901233
CUSTOMER_NAME	: PETER
CUSTOMER_GENDER	: M
ACCOUNT_BALANCE	: 0.0

PLEASE SELECT ONE OPTION FROM BELOW
1.WITHDRAW
2. DEPOSIT
3. CHECK BALANCE
4. EXIT
OPTION: 1
ENTER AMOUNT TO WITHDRAW: 200
AMOUNT WITHDRAWN EXCEEDS THE CURRENT BALANCE!
1.WITHDRAW
2. DEPOSIT
3. CHECK BALANCE
4. EXIT
OPTION: 2
ENTER AMOUNT TO DEPOSIT: 10000
AMOUNT DEPOSITED SUCESSFULLY!!
1.WITHDRAW
2. DEPOSIT
3. CHECK BALANCE
4. EXIT
OPTION: 3
CURRENT BALANCE: 10000
1. WITHDRAW
2. DEPOSIT
3. CHECK BALANCE
4. EXIT
OPTION: 4
THANK YOU FOR BANKING WITH US!!

2. Product and Its type (10 Marks)

You are developing an application for an online shopping platform. You are required to create a class representing a "Product" and its subclasses for different types of products.

Product Class: The **Product** class represents abstract class with the following attributes and methods:

Attributes:

productId (int): A unique identifier for each product.
productName (String): The name of the product.
price (double): The price of the product.

Methods:

Constructor: Initializes the **productId**, **productName**, and **price** attributes.
getProductId(): Returns the product's unique identifier.
getProductName(): Returns the name of the product.
getPrice(): Returns the price of the product.

SmartCliff SmartCliff

displayInfo(): Method is declared as abstract .

SmartCliff SmartCliff

Subclasses of Product:

1. **ElectronicProduct:**

   Attributes:

   warrantyPeriod (int): The warranty period of the electronic product in months.

   Methods:

   Constructor (): Initializes the attributes of the **ElectronicProduct** class, Including calling the superclass constructor.

   getWarrantyPeriod(): Returns the warranty period of the electronic product.

   displayInfo(): Overrides the **displayInfo()** method of the superclass to display information of the product along with the warranty.

2. **ClothingProduct:**

   Attributes:

   size (String): The size of the clothing product (e.g., S, M, L, XL).

   material (String): The material of the clothing product (e.g., Cotton, Silk, Polyester).

   Methods:

   Constructor: Initializes the attributes of the **ClothingProduct** class, including calling the superclass constructor.

   getSize(): Returns the size of the clothing product.

   getMaterial(): Returns the material of the clothing product.

   displayInfo(): Overrides the **displayInfo()** method of the superclass to display additional information about the size and material.

Application Flow:

1. Create a few instances of the **Product** class and its subclasses (**ElectronicProduct** and **ClothingProduct**) with different attributes.
2. Display the information of each product using the **displayInfo()** method.

SAMPLE OUTPUT

ELECTRONIC PRODUCT INFORMATION:

```
Productid        201
ProductName      : Laptop
Price            : $1200.0
Warranty Period  : 12 Months
```

CLOTHING PRODUCT INFORMATION:

```
Productid        301
ProductName      : T-Shirt
Price            : $25.0
Size             : M
Material         : Cotton
```

SmartCliff SmartCliff

3. Employee and Salary (5 Marks)

You are developing a class representing an "Employee" with different functionalities related to salary manipulation. The goal is to use method overloading to provide variations of methods to handle different salary-related scenarios for employees.

Employee Class: The **Employee** class represents an employee with the following attributes and methods:

Attributes:

- **employeeId** (int): A unique identifier for each employee.
- **name** (String): The name of the employee.
- **designation** (String): The job designation of the employee.
- **salary** (double): The salary of the employee.

Methods (Overloaded):

- **Constructor:** Initializes the **employeeId**, **name**, **designation**, and **salary** attributes.
- **displayInfo()**: Displays the basic information of the employee, including name and job designation.
- **increaseSalary(double percentage)**: Overloaded method to increase the salary by the given percentage.
- **increaseSalary(double percentage, double bonus)**: Overloaded method to increase the salary by the given percentage and bonus amount.

Application Flow:

1. Create an instance of the **Employee** class, specifying the employee ID, name, designation, and initial salary.
2. Display the employee's information using the **displayInfo()** method.
3. Increase the employee's salary by a percentage using the **increaseSalary(double percentage)** method.
4. Increase the employee's salary by a percentage and apply a bonus amount using the **increaseSalary(double percentage, double bonus)** method.
5. Display the employee's updated information after each salary manipulation.

SmartCliff SmartCliff